CSCI2010U MIDTERM - SIDE A: CODE YOU CAN WRITE, TRACES, RECURSION, SEARCHING

FIND IT FAST **A1** How to answer · **A2** Java essentials · **A3** Count/min/sum · **A4** Reverse · **A5** Order the steps · **A6** Pseudocode to Java · **A7** Bubble sort · **A8** Searching · **A9** Array insert / delete · **A10** Perfect square · **A11** More whole methods · **A12** Recursion · **A13** 4 steps · **A14** Loops to big-o · **A15** Lab 5

A1 HOW TO ANSWER

Print / return? Make a table: one row per pass or call, one column per variable. Run it line by line.

Write a method: 1) what to return, 2) a variable (count / sum / min), 3) loop or recursion, 4) return AFTER the loop. Test it on the example.

A2 JAVA ESSENTIALS

Array: index from **0**; `a.length` (no brackets); last = `a[a.length-1]`; `a[a.length]` crashes. `int[] a = new int[5];` (all 0) or `{5,2,9}`. Size is fixed.

String: `s.length()`, `s.charAt(i)`, `s.substring(1)` ("hello" gives "ello"), `s.substring(1,3)` ("el", end not included), `s1.compareTo(s2)` (negative = s1 first). Case matters.

`n%10` = last digit, `n/10` drops it (5072: 2 and 507). `i % k == 0` = divisible by k. 7 / 2 = 3, 7 % 2 = 1. `==` compares, `=` assigns.

Defaults: int 0, boolean false, objects null. An int method returns a number (or throws), never null.

Arrays and objects pass a reference copy, so a method changes the SAME array (no return needed). Primitives pass a copy.

A `static` method cannot use instance fields or `this`. Strings are not arrays: `s[i]` fails; `+=` works on Strings, not arrays.

A3 COUNT, MIN, SUM, MULTIPLES (Lab 1)

```
int count = 0;                    // before loop
for (int i = 0; i < a.length; i++) {
  if (a[i] >= target) {   // equal counts
    count++;
  }
}
return count;                     // after loop
```

countAtLeast({25,10,40,25}, 25) = **3**. Above 4 in {5,2,9,2} = **2**.

Min, sum, and multiples of mult from start to end

```
int min = a[0];                   // not 0
for (int i = 1; i < a.length; i++) {
  if (a[i] < min) {
    min = a[i];
  }
}
int sum = 0;
for (int i = 0; i < a.length; i++) {
  sum += a[i];
}
for (int i = start; i <= end; i++) {
  if (i % mult == 0) {        // multiples
    System.out.println(i);
  }
}
```

A4 REVERSE

```
void reverseArray(int[] a) {  // in place
  for (int i = 0, j = a.length - 1;
       i < j; i++, j--) {
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
  }
}                  // {1,2,3,4} -> 4 3 2 1
String reverse(String s) {
  String out = "";
  for (int i = s.length() - 1; i >= 0; i--) {
    out += s.charAt(i);
  }
  return out;                     // "abc" -> "cba"
}
```

A5 ORDER THE STEPS

```
insert: range check, i = a.length-1, while
  i > idx do a[i] = a[i-1], i--; a[idx] = val
binary: low > high NOT_FOUND, mid, equal
  return mid, else recurse on a half
TOH: n == 0 return, TOH(n-1,start,temp,goal),
  print move, TOH(n-1,temp,goal,start)
delete: range check, i = idx+1, then while
  i < length do a[i-1] = a[i]; last = GAP
```

A6 PSEUDOCODE TO JAVA

pseudocode	Java
`for i = a to b`	`for (int i = a; i <= b; i++)`
`for j = 0 to i-1`	`for (int j = 0; j < i; j++)`
`for i = n-1 downto 1`	`for (int i = n-1; i >= 1; i--)`
`A.length = n`	indices 0 to n-1

A7 BUBBLE SORT (L1, L2)

```
for (int i = A.length - 1; i > 0; i--) {
  for (int j = 0; j < i; j++) {
    if (A[j + 1] < A[j]) {
      int temp = A[j];        // swap
      A[j] = A[j + 1];
      A[j + 1] = temp;
    }
  }
}
```

`i` is FIXED during a pass; `j` moves. n elements: n-1 passes. Descending: use `A[j+1] > A[j]`. After pass k the last k are locked. Worst case O(n^2).

start	pass 1	pass 2	pass 3	pass 4
4 2 3	2 3 4	2 3 4		
3 1 2	1 2 3	1 2 3		
5 2 4 1	2 4 1 5	2 1 4 5	1 2 4 5	
8 1 6 4 9	1 6 4 8 9	1 4 6 8 9	1 4 6 8 9	1 4 6 8 9

n	i starts	passes	pass 1 compares	j max
4	3	3	3	2
5	4	4	4	3
6	5	5	5	4

Compares: n(n-1)/2 (5 elements: 10). Sorted {1,2,3}: 0 swaps. Reversed {3,2,1}: 3 swaps.

A8 SEARCHING (L6, L10)

```
for (int i = 0; i < A.length; i++) {
  if (A[i] == val) {
    return i;
  }
}
return -1;        // best O(1), worst O(n)
```

Binary search: array must be SORTED

```
bs(a, val, low, high):
  if (low > high) return NOT_FOUND;
  int mid = (low + high) / 2;
  if (val == a[mid]) return mid;
  else if (val < a[mid])
    return bs(a, val, low, mid - 1);
  else
    return bs(a, val, mid + 1, high);
// start bs(a, val, 0, a.length - 1)  O(log n)
```

a = {1,3,5,7,9,11,13}. Each step: low, high, mid = a[mid].

find	steps	result
11	0,6,3=7 go right; 4,6,5=11	5
3	0,6,3=7 go left; 0,2,1=3	1
5	0,6,3=7; 0,2,1=3; 2,2,2=5	2
4	0,6,3=7; 0,2,1=3; 2,2,2=5, high=1	NOT_FOUND

A9 ARRAY INSERT / DELETE (L10)

```
insert(a, idx, val):
  for (int i = a.length - 1; i > idx; i--) {
    a[i] = a[i - 1];          // shift RIGHT
  }
  a[idx] = val;               // last falls off
delete(a, idx):
  for (int i = idx + 1; i < a.length; i++) {
    a[i - 1] = a[i];          // shift LEFT
  }
  a[a.length - 1] = GAP;
```

example	steps
insert 50 at 1 in 4 6 8 10	4 6 8 8; 4 6 6 8; 4 50 6 8 (2 shifts)
delete idx 1 in 5 6 7 8	5 7 7 8; 5 7 8 8; 5 7 8 GAP (2 shifts)
cost	worst idx 0: O(n); best idx last: O(1)

A10 PERFECT SQUARE (L6, L9)

v0 `while (i <= n)` O(n). v1 `while (i*i <= n)` O(sqrt n). v2 doubles i, then binary search: O(log n).

n	v0 loops	v1 loops	answer
10	10	3 (i=1,2,3)	No
20	20	4 (i=1..4)	No
16	4	4	Yes

A11 MORE WHOLE METHODS

```
boolean isSorted(int[] a) {
  for (int i = 1; i < a.length; i++)
    if (a[i - 1] > a[i]) return false;
  return true;    // empty or 1 element: true
}
int countDivisible(int lo, int hi, int k) {
  int count = 0;
  for (int i = lo; i <= hi; i++)
    if (i % k == 0) count++;
  return count;   // (1, 11, 3) = 3
}
```

A12 RECURSION (L7, Lab 4)

Needs a **base case** (smallest problem, returns) and a **recursive case** (calls itself on a SMALLER problem). No base case = StackOverflowError.

Trace: go DOWN (calls wait), then back UP using the value from below. `sumTo(2)` means the SUM 1+2 = 3, not the number 2. Countdown n..0 = n+1 calls. Print BEFORE the call = going down (3,2,1). Print AFTER = coming up (1,2,3).

sumTo, up, test, sumDigits, countDigits (answers in comments)

```
int sumTo(int n) {
  if (n == 0) return 0;
  return n + sumTo(n - 1);
}             // (0)=0 (1)=1 (2)=3 (3)=6
void up(int n) {              // up(3): 1 2 3
  if (n == 0) return;
  up(n - 1);
  System.out.println(n);
}
void test(int n) {   // test(2): A2 A1 B1 B2
  if (n == 0) return;
  System.out.println("A" + n);
  test(n - 1);
  System.out.println("B" + n);
}
int sumDigits(int n) {
  if (n == 0) return 0;
  return n % 10 + sumDigits(n / 10);
}             // (72) = 2 + (7 + 0) = 9
int countDigits(int n) {
  if (n < 10) return 1;
  return 1 + countDigits(n / 10);
}             // (472) = 1 + (1 + 1) = 3
```

String recursion and call counting

```
String rev(String s) {
  if (s.length() <= 1) return s;
  return rev(s.substring(1))
         + s.charAt(0);
}   // "abc": rev("bc") + a
    // = (rev("c") + b) + a = "cba"
boolean isPal(String t) {
  int n = t.length();
  if (n <= 1) return true;
  if (t.charAt(0) != t.charAt(n - 1))
    return false;
  return isPal(t.substring(1, n - 1));
}   // "abba": a=a, "bb": b=b, "" true
    // "abca": a=a, "bc": b!=c false (2 calls)
int f(int n) {
  if (n <= 1) return 1;
  return f(n - 1) + f(n - 2);
}   // f(2)=2 f(3)=3 f(4)=5; 9 calls for f(4)
```

Towers of Hanoi

```
TOH(n, start, goal, temp):
  if (n == 0) return;
  TOH(n - 1, start, temp, goal);
  print start --> goal;
  TOH(n - 1, temp, goal, start);
call TOH(n, "A", "C", "B");  moves = 2^n - 1
n=2: A-B A-C B-C
n=3: A-C A-B C-B A-C B-A B-C A-C
```

A13 WRITE A RECURSIVE METHOD IN 4 STEPS

```
1 base case: smallest input, return directly
2 one piece of work: n%10, first letter, n
3 call yourself on a SMALLER input:
  n/10, n-1, substring(1)
4 combine: return piece + the call
A loop stop condition = the base case
```

A14 LOOPS TO BIG-O

code	Big-O
`return a[5];`	O(1)
one loop `i<n`	O(n)
two loops one after the other	O(n)
loop inside loop, both to n	O(n^2)
inner `j<i` inside `i<n`	O(n^2)
`while (n>1) n=n/2;`	O(log n)
`for (i=1;i<n;i*=2)`	O(log n)
inner loop of 5, outer to n	O(n)

A15 LAB 5: HOW THE WORK GROWS

approach	work	Big-O
sum each window of k	(n-k+1) x k adds	O(nk)
running sum, add new/drop old	2 ops per step	O(n)
check every pair i < j	n(n-1)/2 pairs	O(n^2)
sorted: index at each end	1 index moves/step	O(n)

O(1) space = scalars only. Use **long** for sums.

CSCI2010U MIDTERM - SIDE B: OOP, INTERFACES, BIG-O, LISTS, EXPLAIN WHY

FIND IT FAST **B1** Inheritance · **B2** Polymorphism · **B3** Explain why · **B4** Compile/runtime · **B5** Abstract class · **B6** Roster class · **B7** compareTo · **B8** Access · **B9** Algorithms · **B10** T(n) worked · **B11** Lists · **B12** Quick facts · **B13** Abstract vs interface

B1 INHERITANCE (L3, L4, Lab 2)

`extends`. Test "is a": Manager IS AN Employee, never the reverse. A subclass can add and **override**.

`super(...)` must be the FIRST line of the constructor (runs the parent constructor).

Parent `private` fields are not visible in the child: use the getter.

In an override, `super.m()` = parent version. Plain `m()` = itself, forever = **StackOverflowError** (runtime).

Pet / Dog / LoudDog

```
class Pet {
  private String name;
  public Pet(String name) { this.name = name; }
  public String getName() { return name; }
  public String sound() { return "..."; }
}
class Dog extends Pet {
  private int tricks;
  public Dog(String name, int tricks) {
    super(name);              // first line
    this.tricks = tricks;
  }
  @Override
  public String sound() { return "Woof"; }
}
class LoudDog extends Dog {
  public LoudDog(String n, int t) {
    super(n, t);
  }
  @Override
  public String sound() {
    return super.sound() + "!";
  }
}
Pet[] p = { new Pet("A"), new Dog("B", 1),
            new LoudDog("C", 2) };
for (Pet x : p) {
  System.out.println(x.sound());
}         // prints:  ...   Woof   Woof!
```

LimitedCounter (Lab 2)

```
class LimitedCounter extends Counter {
  // private int limit;
  public LimitedCounter(String name,
      int limit) {
    super(name);
    this.limit = limit;
  }
  @Override
  public void increment() {
    if (getCount() < limit) {
      super.increment();
    }
  }
}
// Manager: super(salary, name); bonus = 0;
// getSalary: super.getSalary() + bonus
```

B2 POLYMORPHISM (L4)

A parent variable can hold a child object. The version that RUNS is the object's own class.

Read right to left with "is a". A parent variable cannot call child-only methods.

Making an object: declare a REFERENCE (an address), create the object with `new` on the **HEAP**, link them.

code	result
`Pet p = new Dog("B",1);`	OK (Dog is a Pet)
`p.sound()`	runs Dog: Woof
`p.describe()`	COMPILE error
`Dog d = new Pet("A");`	COMPILE error
`Pet[] arr`	holds Pet, Dog, Cat
`void feed(Pet x)`	accepts any Pet child

B3 EXPLAIN WHY

Equal to target counts: "at least" means >=.

super(name) vs super.increment(): the first sets the parent's private state (first line only); the second calls the parent version of an overridden method.

new Person() fails: abstract, cannot be instantiated, even with a constructor.

Comparable and Arrays.sort: sort calls compareTo to decide the order. implements = many roles; extends = one parent.

Checked vs unchecked: unchecked = extends RuntimeException (programmer error, need not be caught or declared): ArrayIndexOutOfBounds, IndexOutOfBounds, NullPointer. Checked = must be caught or declared with `throws`. An **Error** (StackOverflowError) is beyond your control.

B4 COMPILE OR RUNTIME?

situation	error
override calls itself (no `super.`)	RUNTIME StackOverflow
`a[4]` on length 4	RUNTIME ArrayIndex…
`new Shape()` (abstract)	COMPILE
child reads parent private field	COMPILE
`return null` in an int method	COMPILE

B5 ABSTRACT CLASS (L5, Lab 3)

```
abstract class Shape {
  public abstract double area();  // no body
}
class Square extends Shape {
  private double s;
  public Square(double s) { this.s = s; }
  @Override
  public double area() { return s * s; }
}
new Shape()                 // ERROR (abstract)
Shape b = new Square(2);    // OK
class Triangle extends Shape { }  // ERROR
  // fix: write area() OR abstract class
```

Cannot instantiate an abstract class, even with a constructor. A constructor can NOT be abstract. Mixes concrete + abstract methods; can have static methods.

A child must write ALL abstract methods or be abstract itself. Non-abstract = **concrete**.

B6 WRITE A WHOLE CLASS (Roster)

```
class Roster {
  private String[] names;
  private int size;
  public Roster(int capacity) {
    names = new String[capacity];
    size = 0;
  }
  public void add(String name) {
    if (size == names.length)
      throw new IndexOutOfBoundsException();
    names[size] = name;
    size++;
  }
  public String get(int index) {
    if (index < 0 || index >= size)
      throw new IndexOutOfBoundsException();
    return names[index];
  }
  public int size() { return size; }
}
```

B7 INTERFACE AND compareTo

```
class Student implements Comparable<Student> {
  private String name;
  public Student(String name) {
    this.name = name;
  }
  @Override
  public int compareTo(Student o) {
    return name.compareTo(o.name);
  }
}
// Strings use compareTo; numbers use
// Integer.compare(pages, o.pages)
class Employee extends Person
    implements Comparable<Employee> { ... }
Arrays.sort(arr);
// Maya, Alex, Priya -> Alex, Maya, Priya
```

Returns **negative** if this comes first, **0** if equal, **positive** if after. `Arrays.sort` uses it; order does NOT come from insertion order.

Interface = a list of promises (a ROLE); all methods abstract. `implements` MANY, `extends` only ONE (diamond problem: two parents, which method?).

Generics `<T>` restrict the type: `ArrayList<String>` holds Strings only.

B8 ACCESS AND PACKAGES

access	public	protected	none	private
same class	Y	Y	Y	Y
same pkg, subclass	Y	Y	Y	N
same pkg, other	Y	Y	Y	N
diff pkg, subclass	Y	Y	N	N
diff pkg, other	Y	N	N	N

Package: first line `package x;`, class names unique per package, no statement = default package, `java.lang` (System, Math, String) auto-imported, `java.util` has Arrays and Random. Course uses mostly public and private.

B9 ALGORITHMS AND BIG-O (L6 to L9)

Algorithm = finite, deterministic (same input, same output), effective, implementable in any language. Pseudocode is for humans. **Time** = count steps; **space** = memory; time matters most.

T(n) = steps for input size n. Each line costs a constant; `end` and comments not counted. A loop header runs ONE MORE time than its body.

Linear search: best (first spot) O(1), worst (missing) `an+b` = O(n). We use **worst case**: an upper-bound guarantee that happens often.

Ladder, slow to fast: **1, log n, sqrt n, n, n log n, n^2, n^3, 2^n, n!**

Definition: f(n) = O(g(n)) if there are positive c and n0 with 0 <= f(n) <= c*g(n) for all n >= n0. g is an UPPER bound; O(g) is a SET.

Drop constants and small terms: 4n^2+20n+9 = O(n^2); 3+5n^2+7n^3 <= 15n^3 so O(n^3).

True/False rule: simplify, then LEFT = O(RIGHT) is true if left is the same or EARLIER on the ladder.

claim	answer
5n+20 = O(n)	T
n^2 = O(n)	F
n = O(n^3)	T
2^n = O(n^3)	F
3+5n+18n^2 = O(n) / O(n^2)	F / T

B10 T(n) AND BIG-O, WORKED

```
linear search: line 1 for: c1(t+1)
  line 2 if: c2*t | line 3 return i: c3
  line 6 return -1: c4     (t = body runs)
best t=1: 2*c1 + c2 + c3 = constant = O(1)
worst t=n: c1(n+1) + c2*n + c4
  = (c1+c2)n + c1 + c4 = O(n)
3+5n^2+7n^3 <= 3n^3+5n^3+7n^3 = 15n^3
```

B11 LISTS (L11)

Ordered = each element has a position (not sorted by value). Head = start, tail = end.

Interface says WHAT (add, addLast, get, isEmpty, size, toString); the implementation says HOW (ArrList uses an array).

`size` = stored now, `maxSize` = capacity. A bad index throws **IndexOutOfBoundsException** (unchecked, runtime). The slide checks `index > size`; the correct test is `index >= size`.

```
ArrList() { this(DEFAULT_SIZE); }  // 10
ArrList(int capacity) {
  maxSize = capacity;
  size = 0;
  listArray = (T[]) new Object[capacity];
}
get(i): if (i < 0 || i >= size)
    throw new IndexOutOfBoundsException();
  return listArray[i];
add(e, i): shift RIGHT from the end
  for (k = last; k > i; k--)
    listArray[k] = listArray[k - 1];
  listArray[i] = e;
  if (size < maxSize) size++;
addLast(e) when FULL: shift LEFT
  for (k = 1; k < listArray.length; k++)
    listArray[k - 1] = listArray[k];
```

add at head = array INSERT (shift right), O(n). addLast on a full list = shift left, O(n).

B12 QUICK FACTS

expression	result
`Arrays.toString(new int[3])`	[0, 0, 0]
`"Hello".charAt(1)`	'e'
`int a[3];` or `new int[]`	does NOT compile
`7 / 2 + " " + 7 % 2`	3 1

Constructor: `public Person(String name, int age) { this.name = name; this.age = age; }`

Terms: object = state (fields) + behaviour (methods) + identity. Encapsulation = private fields + public methods.

B13 ABSTRACT CLASS VS INTERFACE

	abstract class	interface
keyword	`extends` (only 1)	`implements` (many)
methods	concrete + abstract	all abstract
instantiate?	no	no
fields, ctor	yes	no instance fields
use	siblings share code	a role any class can play